Session 5.2

Meaning in Vectors: Embeddings, Attention, the Transformer

Embed, then contextualize: from one vector per word to one vector per word in its sentence

Based on Chapter 13 of Miao & Feng, *AI for Economists*

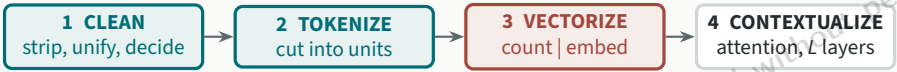
Redistribution prohibited without permission



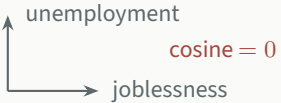
© Zhigang Feng

Zhigang Feng

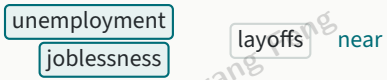
Where the Pipeline Stands



counting: one axis per word



embedding: one point per word



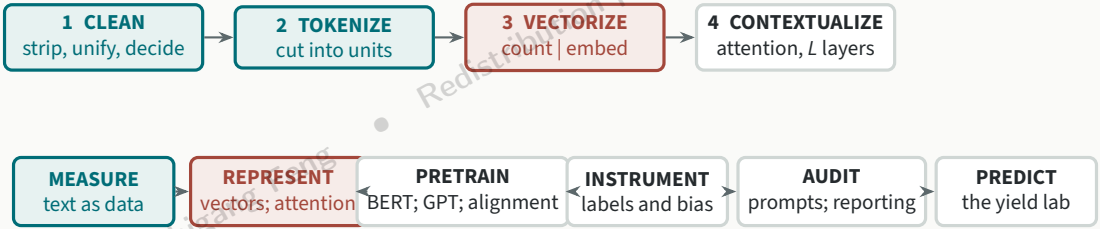
- Deck 5.1 cleaned, tokenized and counted: each document became a **vector**, one entry per vocabulary word.
- Every word is its own axis, so two different words have **cosine** (of their angle) 0: *unemployment* and *joblessness*, *hike* and *raise*.
- **The repair: make words points, where nearness means similar meaning.**

© Zhigang Feng
Redistribution prohibited without permission

Session 5.2

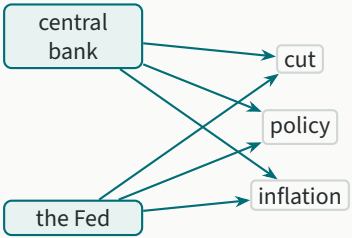
Step 3 · Embed

Words become points, and closeness becomes meaning



© Zhigang Feng
Redistribution prohibited without permission

Know a Word by the Company It Keeps



two words, one set of neighbours: their vectors land close

- Firth (1957): "You shall know a word by the company it keeps," the **distributional hypothesis**.
- *Central bank* and *Fed* both appear near *cut*, *policy* and *inflation*. Nobody declared them related; their contexts did.
- A **distributed representation**: one dense vector $\boldsymbol{u}_w \in \mathbb{R}^d$ per word, $d \ll V$, learned from co-occurrence. Its axes carry no names.
- **Similar context profiles, nearby vectors: the orthogonality of counting is gone.**

© Zhigang Feng Redistribution prohibited without permission

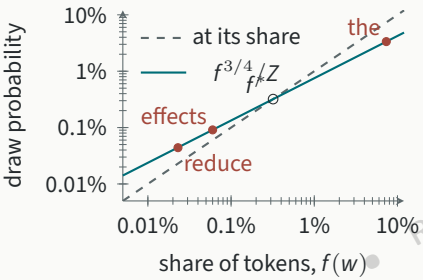
Word2Vec, Step 1: A Window Turns Text Into Pairs



- Each token in turn is the **centre word**; the Δ tokens on each side of it (here $\Delta = 2$) are its **context words**. Each (centre, context) pair is a **real pair**: label 1.
- For every real pair, K **noise words** are drawn at random from the vocabulary and paired with the same centre: label 0. The next frame says how they are drawn.
- **The corpus supplies the data: real pairs against random ones (Mikolov et al. 2013).**

© Zhigang Feng redistribution prohibited without permission

How the Noise Words Are Drawn



- $P_{\text{noise}}(w) = f(w)^{3/4}/Z$ with $Z = \sum_w f(w)^{3/4} = 4.21$ here; $f(w)$: w's share of the 624-word vocabulary's tokens.
- Why not $\propto f$: the noise column fills with *the*, *of*, *to*. Uniform: rare words swamp it.
- $f^{3/4}$ alone never dips below the line; $Z > 1$ pulls it down. Above $f^* = Z^{-4} = 0.32\,\%$ of tokens a word is drawn below its share (*the*: 0.46×), below it above (*reduce*: 1.93×).
- **Negative examples cost nothing: drawn, not labelled. $K = 2$ here, 5–20 in practice.**

© Zhigang Feng
Redistribution prohibited without permission

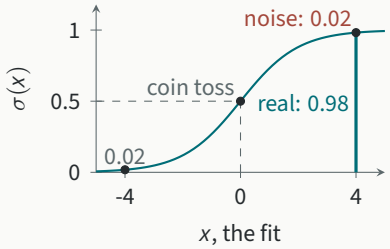
Step 1, Worked: One Centre Word Becomes 12 Rows

real pair	label	its $K = 2$ noise pairs, drawn at random	label
(*raised*, *central*)	1	(*raised*, *next*), (*raised*, *low*)	0
(*raised*, *bank*)	1	(*raised*, *lending*), (*raised*, *3*)	0
(*raised*, *interest*)	1	(*raised*, *in*), (*raised*, *less*)	0
(*raised*, *rates*)	1	(*raised*, *susan*), (*raised*, *earlier*)	0

© Hugo Freund. Redistribution prohibited without permission

- Where we are: the corpus has no labels, so Step 1 makes them. The chapter's sentence, centre *raised*, $\Delta = 2$: four real pairs, label 1 (left column).
- Right column: for each real pair, $K = 2$ words drawn from the FOMC vocabulary by the last frame's rule (seed 42), label 0. Nobody checks them against the text.
- 12 rows from this centre word; then *interest* is the centre, 12 more, and so on through all T tokens: about $T \cdot 2\Delta(1 + K)$ rows, every label free.
- **Next: a classifier that tells the 1s from the 0s, with the word vectors as its only parameters (two frames), fitted one block at a time (the frame after).**

Word2Vec, Step 2: From a Row to a Probability



- A row is a pair (w, c): a centre and a word offered as its context. Its **fit** $x = \tilde{\boldsymbol{u}}_c^\top \boldsymbol{u}_w$ is large when the two vectors point the same way ($\boldsymbol{u}_w$ as centre, $\tilde{\boldsymbol{u}}_w$ as context: $2Vd$ parameters; $\boldsymbol{u}_w$ is kept).
- $\sigma(x) = 1/(1 + e^{-x})$ turns the fit into the chance the pair is real, $P(\text{label } 1) = \sigma(x)$. Noise is the only other label: $P(\text{label } 0) = 1 - \sigma(x)$, the gap up to 1, by symmetry $\sigma(-x)$.
- Read the plot: $x = 0$, a coin toss; $x = 4$, real with chance 0.98, noise 0.02; $x = -4$ the reverse.
- **A binary logit on Step 1's rows; the vectors are its only coefficients.**

© Zhigang Feng
Redistribution prohibited without permission

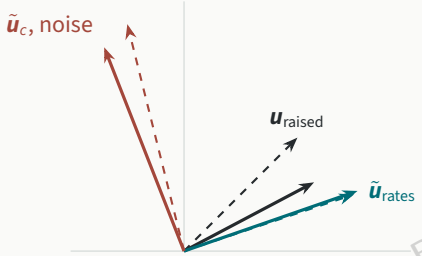
Word2Vec, Step 2: The Objective, Term by Term

$$\max_{\{\boldsymbol{u},\tilde{\boldsymbol{u}}\}} \sum_{t=1}^{T} \sum_{j\neq 0,\ |j|\leq\Delta} \Bigg[\underbrace{\log\sigma\big(\tilde{\boldsymbol{u}}_{w_{t+j}}^{\top}\boldsymbol{u}_{w_t}\big)}_{\text{the real pair: } \log P(\text{label } 1)} + \underbrace{\sum_{k=1}^{K}\log\sigma\big(-\tilde{\boldsymbol{u}}_{c_k}^{\top}\boldsymbol{u}_{w_t}\big)}_{\text{its } K \text{ noise pairs: } \log P(\text{label } 0)} \Bigg]$$

- Inside the brackets, one block of the table: the real pair's log-probability of being real plus its K noise pairs' log-probability of being noise, $c_k \sim P_{\text{noise}}$.
- The real term rises with $x^{+} = \tilde{\boldsymbol{u}}_{w_{t+j}}^{\top}\boldsymbol{u}_{w_t}$: the context's vector is pulled towards the centre's. A noise term rises as $x^{-} = \tilde{\boldsymbol{u}}_{c_k}^{\top}\boldsymbol{u}_{w_t}$ falls: a random word's is pushed away.
- Why both: with real pairs alone, all vectors equal and long would make every $\sigma \to 1$; the noise rows forbid it: near its company, away from random words.
- Outside: t runs over every token as the centre, j over the 2Δ window slots, so over every block the corpus yields; the sum is the log-likelihood of all the labels.
- **Same company, same pulls, same vector: the next frame takes one step.**

© Edgar Fuller • Redistribution prohibited without permission

One Step: Pull the Real Context Closer, Push the Noise Away



dashed: before; solid: after one step (schematic)

- A real pair (raised, rates), $x^+ = \tilde{u}_{\text{rates}}^\top u_{\text{raised}}$; a noise pair (raised, c), $x^- = \tilde{u}_c^\top u_{\text{raised}}$. One step moves u_{raised} by $\eta[(1-\sigma(x^+))\,\tilde{u}_{\text{rates}} - \sigma(x^-)\,\tilde{u}_c]$.
- The real context pulls, hardest while the model doubts the pair ($\sigma(x^+)$ small); the noise pushes, hardest when the model believes it ($\sigma(x^-)$ large).
- The context vectors move by the mirror terms: $\tilde{u}_{\text{rates}}$ towards u_{raised}, $\tilde{u}_c$ away.
- **Same contexts, same pulls: such words end up pointing the same way.**

© Zhigang Feng Redistribution prohibited without permission

Skip-Gram, CBOW and GloVe in Plain Words

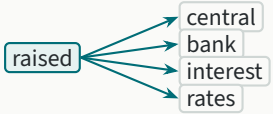


© Zhang Yue Redistribution prohibited without permission

- Skip-gram: show one word, hide its neighbours, let the vectors guess them; a wrong guess nudges the vectors. One lesson per neighbour, so a rare word still gets its own.
- CBOW: show the neighbours, hide the centre word, average their vectors and guess it. One lesson per window: faster, but a rare word's lesson is diluted in the average.
- GloVe: no guessing. Count once how often each pair shares a window, then choose vectors whose dot product matches the log count, frequent pairs weighted more.
- **Three routes, one end: same company, same direction.**

Skip-Gram and CBOW: Same Window, Different Regressor

skip-gram: the centre predicts each context word



CBOW: the window's mean predicts the centre



skip-gram, one row per pair (t, j): $\log \sigma(\tilde{\boldsymbol{u}}_{w_{t+j}}^\top \boldsymbol{u}_{w_t}) + \sum_k \log \sigma(-\tilde{\boldsymbol{u}}_{c_k}^\top \boldsymbol{u}_{w_t})$

CBOW, one row per window t, $\bar{\boldsymbol{h}}_t = \frac{1}{2\Delta} \sum_{j \neq 0} \boldsymbol{u}_{w_{t+j}}$: $\log \sigma(\tilde{\boldsymbol{u}}_{w_t}^\top \bar{\boldsymbol{h}}_t) + \sum_k \log \sigma(-\tilde{\boldsymbol{u}}_{c_k}^\top \bar{\boldsymbol{h}}_t)$

	skip-gram	**CBOW**
predicting side	the centre word, $\boldsymbol{u}_{w_t}$	the window's mean, $\bar{\boldsymbol{h}}_t$
predicted side	each context word, $\tilde{\boldsymbol{u}}_{w_{t+j}}$	the centre word $\tilde{\boldsymbol{u}}_{w_t}$, or a noise word
rows per window	2Δ real $+ 2\Delta K$ noise	1 real $+ K$ noise: 2Δ times fewer, so faster
one step moves	the pair's two vectors, in full	one step, spread over the 2Δ context words

- **Same σ, same logit, same window; only what predicts differs: one word or the window's mean. Skip-gram: a rare word's every occurrence is its own row.**

© Zhigang Feng Redistribution prohibited without permission

GloVe: Count First, Then Fit the Table

$$\log N_{vv'} \approx b_v + \tilde{b}_{v'} + \boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$$

	$\tilde{b}_{v'}$	$\tilde{b}_{v'}$	$\tilde{b}_{v'}$	$\tilde{b}_{v'}$
b_v	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$
b_v	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$
b_v	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$

one cell per word pair (Pennington et al. 2014)

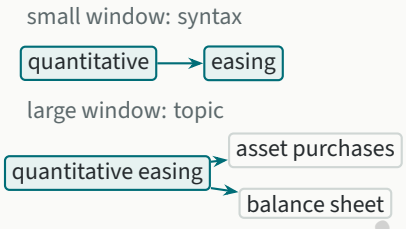
- What we have: the $V \times V$ table $N_{vv'}$, how often v' falls within v's window, the one B1 builds. Word2vec never writes it down; GloVe fits it.
- A gravity equation, trade = size × size × closeness: b_v, $\tilde{b}_{v'}$ absorb each word's frequency, the dot product how much more than chance the two meet (B1's PMI).

$$\min_{\boldsymbol{u},\tilde{\boldsymbol{u}},b,\tilde{b}} \sum_{v,v'} h(N_{vv'}) \left(b_v + \tilde{b}_{v'} + \boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'} - \log N_{vv'}\right)^2$$

- Weighted least squares over the cells: $h(0) = 0$ drops the empty cells, and h is capped, so the commonest pairs do not dominate (the chapter's two roles of h).
- **Three routes to one object: word2vec streams the rows, GloVe fits the count table, B1 factorises it by SVD; all reward words that keep the same company.**

© Zhiyuan Feng. Redistribution prohibited without permission

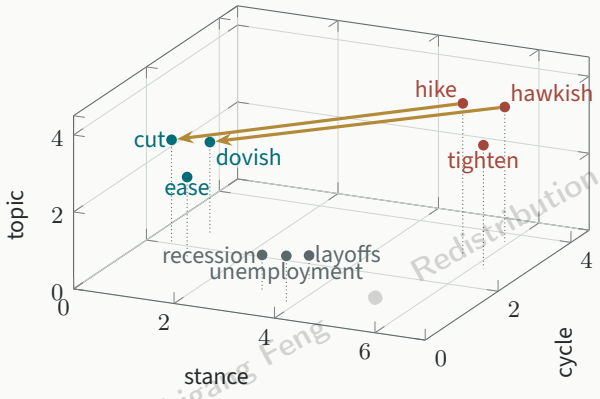
Window, Dimension, Domain



- Small windows ($\Delta \leq 3$) find syntax; large ones ($\Delta \geq 10$) find topic, usually what a measure wants. The vector length *d* is our choice too, usually 100–300.
- Domain matters more: trained on general text, *discount* sits near shop markdowns, not the discount rate; *yield* near harvests, not bond yields.
- **Train on the corpus you measure.**

© Zhigang Feng Redistribution prohibited without permission

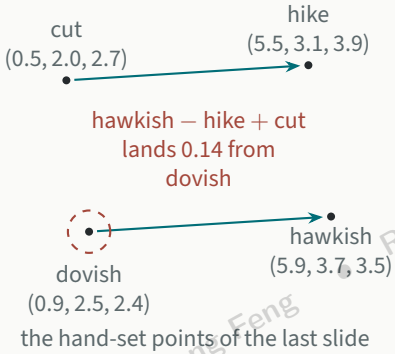
The Geometry: Clusters, Parallel Moves, Coupled Axes



© Zhigang Feng
Redistribution prohibited without permission

- Nine points, hand-set on named axes; real vectors have 100–300 unnamed ones.
- **Clusters**: tightening (red), easing (teal), the cycle (grey).
- **Parallel moves**: hike→cut and hawkish→dovish point the same way (gold).
- **Coupling**: *hike* is high on stance *and* topic.
- **A direction can mean something; an axis rarely does.**

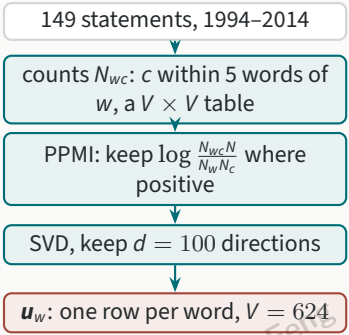
Analogies, Worked, and Why the Metric Is Cosine



© Zhigang Feng
Redistribution prohibited without permission

- **Parallelogram rule:** $\boldsymbol{u}_A - \boldsymbol{u}_B \approx \boldsymbol{u}_C - \boldsymbol{u}_D$. Here hike − cut = (5.0, 1.1, 1.2) and hawkish − dovish = (5.0, 1.2, 1.1): cosine 0.9996.
- hawkish − hike + cut = (0.9, 2.6, 2.3), 0.14 from dovish. The famous case: king − man + woman ≈ queen.
- $\|\boldsymbol{u} - \boldsymbol{u}'\|^2 = \|\boldsymbol{u}\|^2 + \|\boldsymbol{u}'\|^2 - 2\|\boldsymbol{u}\|\|\boldsymbol{u}'\| \cos\theta$: distance also reads the lengths, and length tracks frequency.
- **Compare directions with cosine, not lengths with distance.**

Lab B1: Word Vectors From the FOMC Statements, the Setup



- The question: what do the Committee's own words mean, judged by their company in its statements? Vectors trained on news would answer for news.
- 149 statements are too few for skip-gram, so lab cell B1 counts, on words seen $\geq$10 times.
- PMI: how much more often w and c meet than chance (N: all pairs). Then $\mathrm{PPMI} \approx \mathbf{U}_d \boldsymbol{\Sigma}_d \mathbf{V}_d^\top$; $\boldsymbol{u}_w$ = row w of $\mathbf{U}_d \boldsymbol{\Sigma}_d^{1/2}$, length 1.
- **A close cousin of SGNS, whose optimum has $\tilde{\boldsymbol{u}}_c^\top \boldsymbol{u}_w = \mathrm{PMI}(w, c) - \log K$ (Levy & Goldberg 2014).**

© Zhigang Feng · Redistribution prohibited without permission

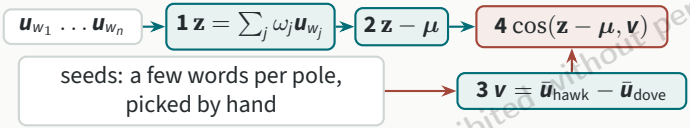
What the FOMC Vectors Learned

word	its five nearest neighbours by cosine				
inflation	expectations	longer	pressures	term	stable
employment	maximum	objectives	seeks	foster	toward
committee	today	decided	open	raise	particular
accommodation	remove	can	removed	begin	take

© Zhiwei Feng. Unauthorized distribution prohibited without permission

- Read the neighbours as phrases: *inflation* meets *expectations*, *longer* and *pressures* in "longer-term inflation expectations" and "inflation pressures"; *maximum employment* is the mandate's. A 5-word window learns set phrases.
- "raise : raised :: lower : ?" has no answer: *raised* occurs fewer than 10 times and gets no vector — the method, not the geometry.
- In B1: print other words' neighbours; change the window or minimum count; see what survives.
- **Read the neighbours before trusting a direction.**

From Word Vectors to a Document Score: The Recipe



© Zbigniew Pastuszak. Free of redistribution prohibited without permission

- Seeds, the only hand input: a few words that mean one pole and nothing else, picked from the statements or a published list, and reported. Here *hike, tighten, hawkish* / *cut, ease, dovish*; B2: *raise, increase, increased, increasing, tightening, firming* / *lower, easing, eased, accommodative, accommodation*.
- **1** $\mathbf{z}$: the document's word vectors averaged, weights ω_j preset ($1/n$; TF-IDF in B2).
- **2** $\boldsymbol{\mu} = \frac{1}{V}\sum_w \boldsymbol{u}_w$, the average word; $\mathbf{z} - \boldsymbol{\mu}$ is what the document says beyond it.
- **3** $\boldsymbol{v}$: the hawkish seeds' mean minus the dovish seeds' mean, pole to pole. **4** score $= \cos(\mathbf{z} - \boldsymbol{\mu}, \boldsymbol{v})$: $+1$ hawkish, -1 dovish, 0 neither.
- **Seeds, weights and centring are three choices, and the score is a regressor: validate it like one (Gennaro & Ash 2022; Ash & Hansen 2023).**

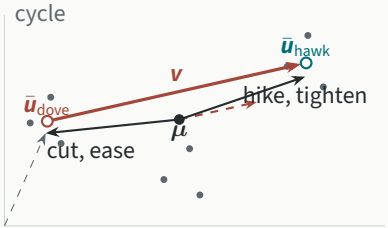
Reading the Picture: Words Are Points, a Document Is a Point, Stance Is an Arrow



- Every word is a point; a document is the average of its words' points, one point $\mathbf{z}$. The average of all words, $\boldsymbol{\mu}$, is the centre of the cloud.
- Two hand-picked lists give two poles: the hawkish words' centre and the dovish words' centre. The arrow from one to the other is the stance direction $\boldsymbol{v}$.
- Score $\mathbf{z}$: draw its arrow from $\boldsymbol{\mu}$; $+1$ if it runs along $\boldsymbol{v}$ (hawkish), -1 against (dovish), 0 at right angles (something else).
- **Measure from the centre, along the pole-to-pole arrow.**

© Zhigang Feng
Redistribution prohibited without permission

The Score on the Plane: the Average Word, Two Poles, One Arrow

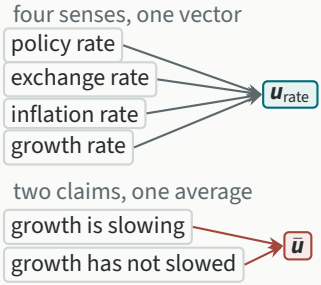


document	uncentred	centred
hike, tighten	$+0.94$	$+0.97$
cut, ease	$+0.54$	-0.96

- The same picture with numbers: nine hand-set points, μ their mean; the poles $\bar{u}_{\text{hawk}}$, $\bar{u}_{\text{dove}}$ are the seed lists' means; v joins them.
- *hike, tighten*: its arrow from μ runs nearly along v (dashed red, copied to μ), cosine $+0.97$; *cut, ease*: against it, -0.96.
- Uncentred, arrows start at the origin, where every word lies in v's quadrant (dashed grey): *cut, ease* scores $+0.54$, the wrong sign.
- **Centre first: a score is a direction relative to the average word.**

© Zhigang Feng
Redistribution prohibited without permission

Four Limits of One Vector per Word



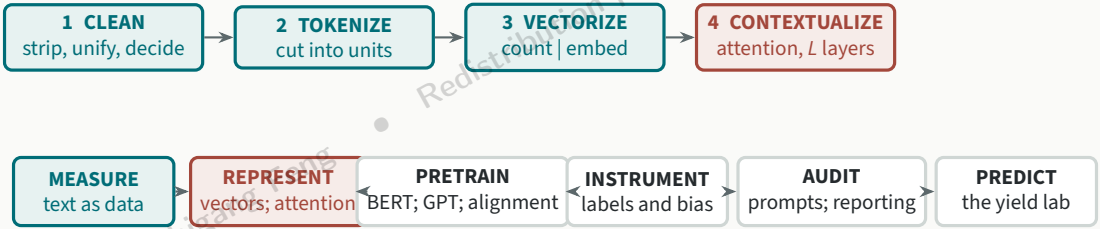
- **Polysemy.** *rate*: policy, exchange, inflation or growth; one vector averages the senses.
- **Negation.** "growth is slowing" and "growth has not slowed" average to one point.
- **Corpus dependence.** *Mask* meant face before 2020, mandate after: a language shift breaks a series.
- **Inherited bias.** man : programmer :: woman : homemaker (Bolukbasi et al. 2016; Caliskan et al. 2017).
- **All four come from one vector per word, whatever the sentence around it.**

© Zhigang Feng
Redistribution prohibited without permission

Session 5.2

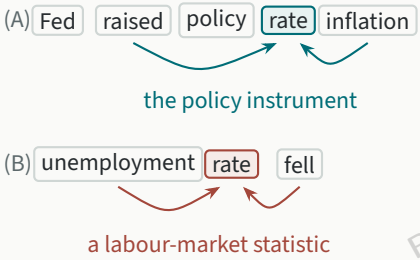
Step 4 · Contextualize

A weight computed from the sentence, instead of fixed before it is read



© Zhigang Feng
Redistribution prohibited without permission

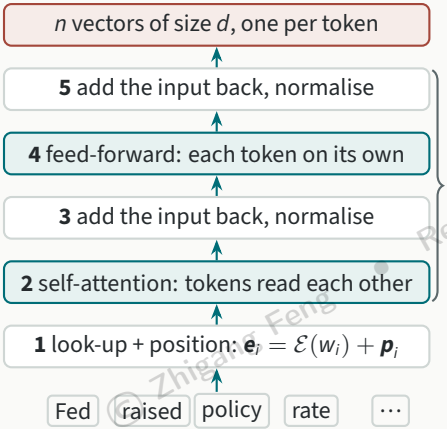
One Word, Two Sentences



- (A) The Fed raised its policy *rate* in response to persistent inflation. (B) The unemployment *rate* fell to its lowest level in decades.
- One token, two economic objects: an instrument the Fed sets, and a labour-market statistic.
- A static vector gives both the same $\boldsymbol{u}_{\text{rate}}$; a document vector with fixed ω cannot separate them.
- **The weights must be computed from the sentence itself.**

© Zhigang Feng
Redistribution prohibited without permission

The Transformer Layer at a Glance

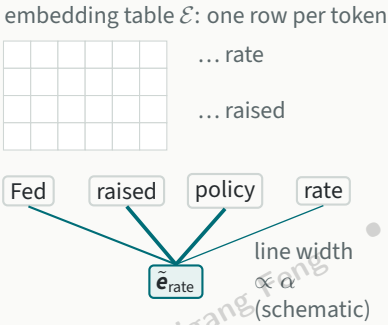


© Zhigang Feng
Redistribution prohibited without permission

- In: n tokens. Out: n vectors of size d, each now carrying its sentence.
- Only step 2 moves information between positions; the others work on one token's vector.
- Steps 3 and 5 keep each vector and rescale it; step 4 transforms it with one small shared network.
- **Steps 2–5 repeat L times (12 in multilingual BERT). The next frames open each box.**

The Idea: Re-Weight the Sentence for Each Word

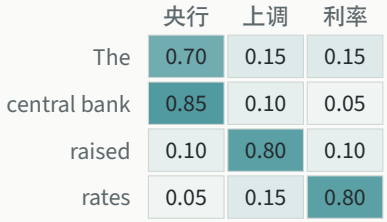
look-up + position → **self-attention** → add & normalise → feed-forward → add & normalise $\times L$



- The **embedding table** $\mathcal{E}$: one learned vector per token ID, plus a position vector: $\boldsymbol{e}_i = \mathcal{E}(w_i) + \boldsymbol{p}_i$.
- Rebuild token i: $\tilde{\boldsymbol{e}}_i = \sum_j \alpha_{ij}\, \mathcal{E}(w_j)$; w_j is token j, $\alpha_{ij} \geq 0$ its weight, $\sum_j \alpha_{ij} = 1$.
- The ω_j were fixed before reading; the α_{ij} come from *this* sentence: (A) *rate* leans on raised and policy, (B) on unemployment and fell.
- **The same token, different neighbours read, a different vector: self-attention.**

© Zhigang Feng
Redistribution prohibited without permission

Attention Began as Alignment in Translation

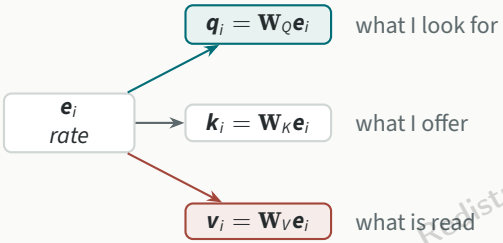


rows: the English word being written; columns: the Chinese source (central bank, raised, interest rate). Illustrative weights

- A translator who reads the whole Chinese sentence, closes the book and writes from memory: the recurrent network, one state holding everything.
- A translator who, for each English word, looks back at the source word that matters: attention (Bahdanau et al. 2015); the table is where the eye rests: **cross-attention**.
- The Transformer keeps the looking back and drops the memory state (Vaswani et al. 2017).
- **Self-attention is this looking back, turned on one sentence.**

© Zhigang Feng Redistribution prohibited without permission

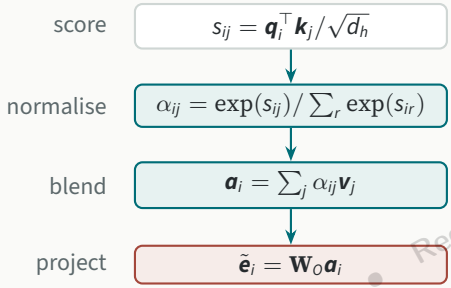
Query, Key, Value: Three Roles for One Token



- Three learned matrices $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d_h \times d}$ give each token three roles, request, index label, content: a soft database look-up.
- In (A) the query of *rate* meets the keys of Fed, raised, policy and inflation; in "the discount rates applied to pension funds", *rates* meets discount and pension.
- **Three separate matrices, so attention is asymmetric: *i* may read *j* heavily while *j* barely reads *i*.**

© Zhigang Feng
Redistribution prohibited without permission

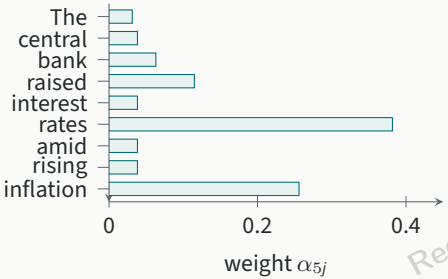
Score, Normalise, Blend, Project



- Sizes: $\boldsymbol{q}_i, \boldsymbol{k}_j, \boldsymbol{v}_j \in \mathbb{R}^{d_h}$; s_{ij} one number; $\mathbf{W}_O \in \mathbb{R}^{d \times d_h}$ returns $\tilde{\mathbf{e}}_i$ to size d.
- The softmax makes each row a distribution: position i spends one unit of attention on the value vectors.
- Why $\sqrt{d_h}$: the variance of $\boldsymbol{q}^\top \boldsymbol{k}$ grows with d_h; unscaled, the softmax piles onto one position and the gradients elsewhere vanish.
- **$\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$ and $\mathcal{E}$ are learned; α_{ij} is computed for every input: a share, never a coefficient.**

© Zhigang Feng
Redistribution not permitted without permission

Example 1: Scores to Weights for the Query *interest*



token j	s_{5j}	$e^{s_{5j}}$	α_{5j}
The	-0.2	0.82	0.03
central	0	1.00	0.04
bank	0.5	1.65	0.06
raised	1.1	3.00	**0.11**
interest	0	1.00	0.04
rates	2.3	9.97	**0.38**
amid	0	1.00	0.04
rising	0	1.00	0.04
inflation	1.9	6.69	**0.26**

Redistribution prohibited without permission
© Yingcai Feng

- s_{5j}: query token 5, *interest*, scored against token j, set by hand here ($\boldsymbol{q}_5^\top \boldsymbol{k}_j/\sqrt{d_h}$ once trained); $e^{s_{5j}}$ makes it positive; $\alpha_{5j} = e^{s_{5j}} / \sum_r e^{s_{5r}}$, its share of the sum, 26.13.
- **64% of the attention goes to *rates* and *inflation*: this *interest* is an interest rate, not a stake.**

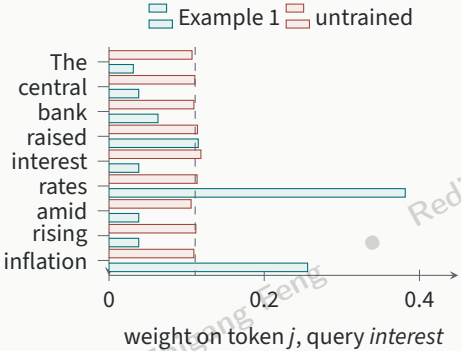
Example 2: Blend, Project, Add

		level	up	prices
rates	$0.38\times$	1	0.3	0
inflation	$0.26\times$	0	0.2	1
raised	$0.11\times$	0	1	0
rising	$0.04\times$	0	0.5	0.2
bank	$0.06\times$	0.5	0	0
$+\cdots=\boldsymbol{a}_5$		0.46	0.30	0.26
$\boldsymbol{e}_{\text{interest}}$		1	0	0
$\boldsymbol{e}+\tilde{\boldsymbol{e}}$		1.46	0.30	0.26

- Value vectors get three named coordinates: rate level, up, price pressure (illustrative; real ones are unnamed).
- Blend with Example 1's weights: $\boldsymbol{a}_5=\sum_j \alpha_{5j}\boldsymbol{v}_j=(0.46,\ 0.30,\ 0.26)$; with $\mathbf{W}_O=\mathbf{I}$ here, $\tilde{\boldsymbol{e}}_5=\boldsymbol{a}_5$.
- Add the token's own vector: $(1.0,\ 0.0,\ 0.0)+(0.46,\ 0.30,\ 0.26)=(1.46,\ 0.30,\ 0.26)$. Two *rate* tokens, identical before, now differ by what each read.
- **Same size, new content: *interest* now says "going up" and "price pressure".**

© Zhigang Feng
Redistribution prohibited without permission

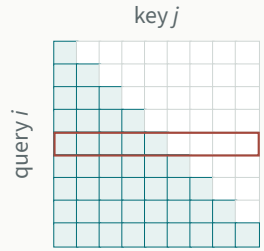
Example 3: The Same Arithmetic, Untrained



© Zhaoyang Cheng
Redistribution prohibited without permission

- The same tokens through an untrained head: random embeddings ($d = 16$) and random $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ ($d_h = 8$).
- Its row for *interest*: 0.106–0.118, against $1/9 = 0.111$. Nearly flat.
- Training (Deck 5.3) fits $\mathbf{W}_Q, \mathbf{W}_K$ so that the keys that matter score high. Lab C1 reproduces Example 1; C1b, this head.
- **The arithmetic is fixed; the pattern is learned.**

The Causal Mask: A Writer Must Not Read Ahead

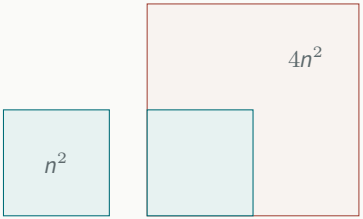


before	.03	.04	.06	.11	.04	.38	.04	.04	.26
masked	.11	.13	.22	.40	.13	0	0	0	0

- Goal: a generator writes token $i+1$ from tokens $1, \ldots, i$ (Deck 5.3). Trained unmasked, position i could read the answer and copy it.
- Set $s_{ij} = -\infty$ for $j > i$: $\alpha_{ij} = 0$, and each row renormalises over $j \leq i$: the **causal mask**.
- Example 1 masked: the sum falls to 7.47; *raised* takes 0.40; *rates* and *inflation* get 0. Lab C1b masks the untrained head.
- **An encoder reads a finished text: every token reads every other, the square. A decoder is writing: a token reads only what is on the page, the triangle.**

© Zhigang Feng · Redistribution prohibited without permission

What the Square Costs

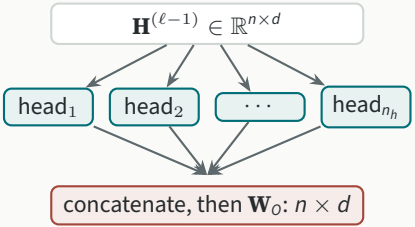


text	tokens n	scores n^2
the nine-token sentence	9	81
median FOMC statement	410	168,100
longest statement	1,053	1,108,809
100 statements	41,000	1,681,000,000

- Every query scores every key: n tokens give n^2 scores per head and layer, an $n \times n$ table (counts: the lab's 245 statements, GPT-4's tokenizer).
- Doubling n quadruples work and memory, so a long filing is cut into blocks that cannot see each other, severing Item 7 from Item 1A: Deck 5.3's window limit.
- **The square is the price of letting every token read every other.**

© Zhigang Feng Redistribution prohibited without permission

Many Heads, and the Whole Sentence at Once



$4n_h d_h d = 4d^2$ parameters, whatever n_h is

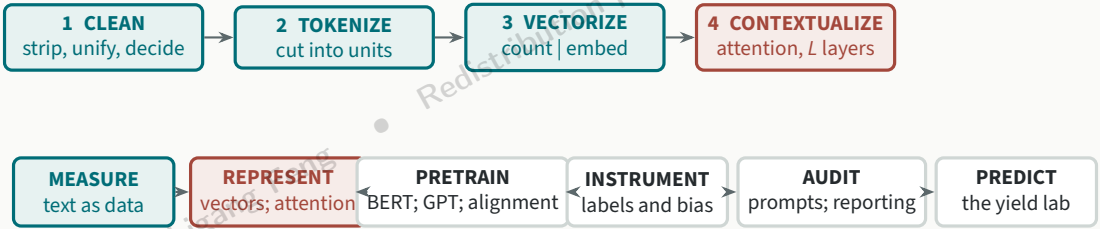
- Rows $\mathbf{H} \in \mathbb{R}^{n\times d}$. Head h has its own $\mathbf{W}_Q^h, \mathbf{W}_K^h, \mathbf{W}_V^h$, so $\mathbf{Q}_h = \mathbf{H}\mathbf{W}_Q^h$ etc., and computes $\mathrm{softmax}(\mathbf{Q}_h\mathbf{K}_h^\top/\sqrt{d_h})\,\mathbf{V}_h$, $d_h = d/n_h$.
- Why they differ: different random starts, then training moves each to the pattern that lowers the loss the others miss. Nothing assigns the jobs.
- Readers of one set of minutes: inflation, the labour market, references back.
- **A head weights positions; it does not split a document into topics.**

© Zhigang Feng
Redistribution prohibited without permission

Session 5.2

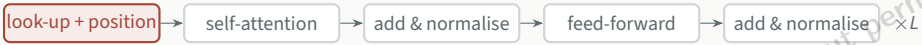
One Transformer Layer

Position, residual connections, normalisation, and the feed-forward block



© Zhigang Feng
Redistribution prohibited without permission

Attention Cannot See Word Order



the same tokens, opposite claims

growth exceeded inflation

inflation exceeded growth

queries and keys see tokens, not places

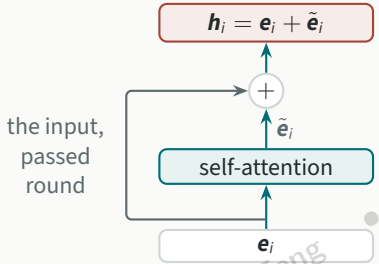
$$\boldsymbol{e}_i = \mathcal{E}(w_i) + \boldsymbol{p}_i$$

- Without position vectors attention is permutation-invariant: shuffle the tokens and no score changes (a causal mask leaks some order; Haviv et al. 2022).
- **Sinusoidal**: $p_{i,2l} = \sin(i/10000^{2l/d})$, $p_{i,2l+1} = \cos(i/10000^{2l/d})$.
- Learned positions stop at the training length; RoPE (Su et al. 2024) with scaling and ALiBi (Press et al. 2022) read past it.
- **Order is not in the mechanism; it is added to the input.**

© Zhigang Feng
Redistribution prohibited without permission

Residual Connection: Keep the Token, Add What It Read

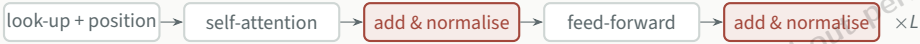




- Output = input + update: the sum keeps the word as well as what it read. “Residual” names the update, not a regression residual or Session 2’s Euler residual.
- $\partial \boldsymbol{h}_i / \partial \boldsymbol{e}_i = \mathbf{I} + \partial \tilde{\boldsymbol{e}}_i / \partial \boldsymbol{e}_i$: the $\mathbf{I}$ carries the gradient through all L layers; without it one Jacobian per layer shrinks it, the **vanishing gradient**.
- **A layer with nothing to add learns $\tilde{\boldsymbol{e}}_i \approx 0$ and passes its input on.**

© Zhigang Feng
Redistribution prohibited without permission

Layer Normalisation: Standardise Each Token's Vector



one token's vector, $d = 4$

2	4	6	8

$(\boldsymbol{x} - \mu)/\sigma$

$\mu = 5, \sigma = 2.24$

-1.34	-0.45	0.45	1.34

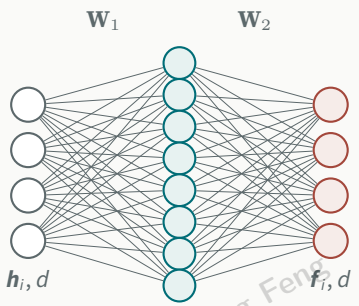
then $\boldsymbol{\gamma} \odot (\cdot) + \boldsymbol{\beta}$, learned per coordinate

- $\mathrm{LN}(\boldsymbol{x}) = \boldsymbol{\gamma} \odot (\boldsymbol{x} - \mu)/\sqrt{\sigma^2 + \epsilon} + \boldsymbol{\beta}$: μ, σ over the d entries of *one* token's vector; $\boldsymbol{\gamma}, \boldsymbol{\beta}$ learned.
- Why: each residual adds to the vector, so its scale drifts layer by layer; LN puts every layer's input on one scale, like standardised regressors.
- The strip and the layer picture show the original order, add then normalise; Machine B and most current models normalise before each block.
- **Neither the residual nor LN moves information between tokens. Only attention does.**

© Zhigang Feng
Redistribution prohibited without permission

The Feed-Forward Block: Transform Each Token on Its Own

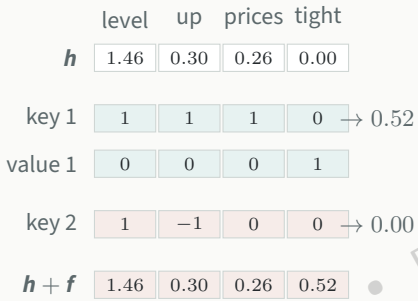
look-up + position → self-attention → add & normalise → **feed-forward** → add & normalise $\times L$



- Attention only averages values, a linear mix; combining *rate*, *raised*, *inflation* into "tightening" needs a nonlinear step.
- $\boldsymbol{f}_i = \mathbf{W}_2\,\mathrm{ReLU}(\mathbf{W}_1\boldsymbol{h}_i + \boldsymbol{b}_1) + \boldsymbol{b}_2$, $\mathbf{W}_1 \in \mathbb{R}^{4d\times d}$, $\mathbf{W}_2 \in \mathbb{R}^{d\times 4d}$: Session 2's one-hidden-layer network.
- Same weights at every position; output size d, added back. Machine B uses GELU, a smooth ReLU.
- **Attention moves information between tokens; this block transforms each one.**

© Zhigang Feng Redistribution prohibited without permission

Example: A Hidden Unit as a Key–Value Memory



© Zhigang Feng

Redistribution prohibited without permission

- Illustrative coordinates: rate level, up, price pressure, tightening; ***h*** from Example 2; bias -1.5.
- Unit 1's key (a row of $\mathbf{W}_1$): $2.02 + (-1.5) = 0.52$. It fires on "a rate going up with price pressure" and adds $0.52\times$ its value (a column of $\mathbf{W}_2$).
- Unit 2 looks for "a rate going down": -0.34, ReLU 0, silent. The vector now says "tightening", which no token said.
- **Rows of $\mathbf{W}_1$ detect patterns; columns of $\mathbf{W}_2$ write back what goes with them (Geva et al. 2021).**

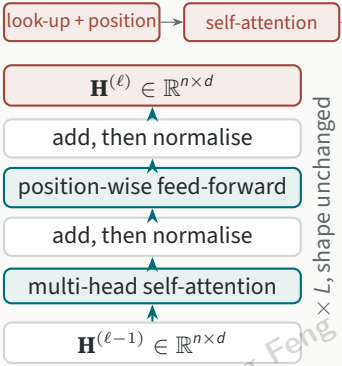
Where the Parameters Are: Machine B, Part by Part

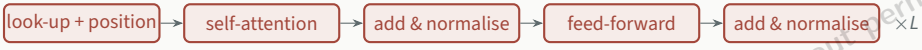
part of Machine B	count	share
feed-forward $\mathbf{W}_1$, 3 layers	198,144	31.4%
feed-forward $\mathbf{W}_2$, 3 layers	196,992	31.2%
attention $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$, 3 layers	148,608	23.5%
attention $\mathbf{W}_O$, 3 layers	49,536	7.8%
position table, 128 positions	16,384	2.6%
character table, 78 characters	9,984	1.6%
output layer, 78 characters	9,984	1.6%
layer norms before attention	768	0.1%
layer norms before feed-forward	768	0.1%
final layer norm	256	0.0%
whole model	631,424	100%

- **Machine B**: Session 1's character-level text generator, trained on the 245 FOMC statements less every 20th; 3 layers, 4 heads, $d = 128$.
- Per layer, feed-forward $8d^2$ against attention's $4d^2$: 62.6% of the model. Lab C2 prints this table.
- **Most parameters sit in the feed-forward blocks, where associations learned from text are kept.**

© Zhigang Feng
Reproduction prohibited without permission

One Layer, Stacked L Times

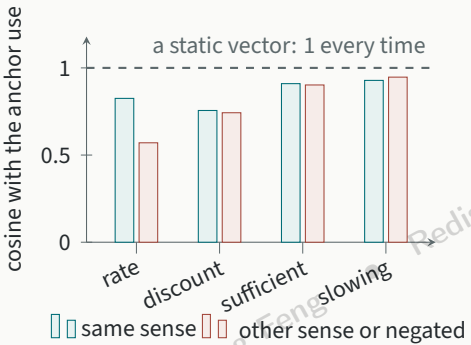




- The layer's output has the input's shape, $n \times d$, so layers stack with no change of interface.
- Shallow layers lean to local syntax (*quantitative easing*); deep ones link paragraphs, a "risk factor" early in an annual report with a "regulatory change" later.
- Machine B stacks 3 layers, $d = 128$; multilingual BERT 12, $d = 768$.
- **$\mathbf{H}^{(L)}$ is where Deck 5.3 starts.**

© Zhigang Feng · Redistribution prohibited without permission

What Context Buys, Measured



© Zhigang Feng — Redistribution prohibited without permission

- Multilingual BERT (12 layers, $d = 768$), last layer: one word in three sentences, cosine with its anchor use.
- *rate* separates its senses, 0.825 vs 0.570 (unemployment rate); *discount* barely, 0.756 vs 0.743 (a price discount).
- Negation barely moves it: *sufficient* 0.910 vs 0.902; for *slowing* the negated use is the closer, 0.947 vs 0.928.
- **Context can fix a sense; whether it does, and whether it reads "not", must be measured.**

Reading 10-Ks by Paragraph Attention: Design and Findings

one 10-K filing, split into paragraphs

↓

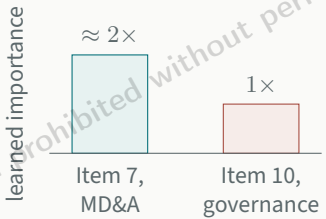
embed each: 3,072 numbers; keep 64

↓

self-attention over paragraphs, then pool

↓

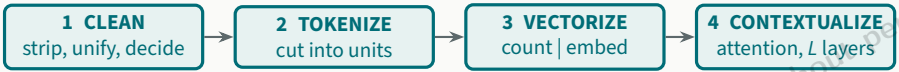
sign of the $[-1, +30]$ return



© Zhigang Feng. No distribution or reproduction prohibited without permission

- Kim, Muhn, Nikolaev & Zhang (2024) make the paragraph the "token", one vector each: 76,929 filings, 2004–2024, 20,712,462 paragraphs.
- Target: the sign of the return from a day before the filing to 30 days after. Attention beats both attention-free baselines; MD&A gets almost twice governance's weight.
- Learned to predict returns, so "important" means useful, not causal; template text repeated in every filing may spread α_{ij} thin.
- **Attention routes information inside the text it is given. It does not retrieve truth.**

The Whole Pipeline



rung	"rates" becomes	size	similar words	senses	you can read
counts, TF-IDF	one column of the matrix	$V \sim 10^4$–10^5, mostly zeros	orthogonal	one per word	each axis is a word
word vector	one point $\boldsymbol{u} \in \mathbb{R}^d$	$d = 100$–300, dense	close	one per word	no axis is a word
contextual vector	a point per sentence	$d = 768$ in multilingual BERT	close	one per use	the weights, not the meaning

© Zhitang Feng
Redistribution prohibited without permission

- Report every step: the cleaning script; the tokenizer and its version; the vector method, corpus and period; the model and its version.
- **Each rung buys meaning and sells the audit chain: pick the rung the design needs.**

Lab, Parts B and C: Vectors and Attention on the FOMC Statements

cell	what you run	the question you answer
B1	word vectors by PPMI and SVD	Which neighbours are phrases? Which survive another window?
B2	the hawk–dove direction	Which seeds survive, and what does the score bundle in?
B3	validation on the policy path	Where does the embedding score fail?
C1, C1b	Example 1; the untrained head, masked	What does the mask set to 0? How flat is the row?
C2–C3	Machine B's parameters and maps	Where are the parameters? Is the upper triangle 0?



- **Each cell reproduces the slide numbers it names; C4–C5 belong to Deck 5.3.**

© Zhaoning Feng · Redistribution prohibited without permission